



nextwork.org

APIs with Lambda + API Gateway

CH

Arpita Chowdhury

Create method

Method details

Method type

GET

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.


☐ **HTTP**
Integrate with an existing HTTP endpoint.


☐ **Mock**
Generate a response based on API Gateway mappings and transformations.


☐ **AWS service**
Integrate with an AWS Service.


☐ **VPC link**
Integrate with a resource that isn't accessible over the public internet.


☒ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

Response transfer mode [Info](#)

☒ **Buffered**
Wait to receive the complete response before beginning transmission.



Introducing Today's Project!

To set up for my project, I created a secret called TopSecretInfo that contains sensitive information stored securely in AWS Secrets Manager, which I will later access to generate logs and test my monitoring and alerting setup.

Tools and concepts

Services I used were AWS Lambda, Amazon API Gateway, Amazon DynamoDB, AWS IAM, and AWS CloudWatch. Key concepts I learnt include Lambda functions for running backend code without managing servers, integrating Lambda with API Gateway to build REST APIs, retrieving data from DynamoDB using query parameters, deploying APIs to stages, handling request and response mappings, and documenting APIs in JSON for production-ready usage.

Project reflection

This project took me approximately 2 hours. The most challenging part was configuring and connecting all the serverless components correctly, especially integrating Lambda with API Gateway and troubleshooting request paths and parameters. It was most rewarding to successfully deploy a working API and see real data returned through a public endpoint, knowing it was built without managing any servers.



Arpita Chowdhury

NextWork Student

nextwork.org

I chose to do this project today because I wanted hands-on experience building a real serverless API, which is a core skill for cloud and backend roles, and I wanted to better understand how AWS services like Lambda and API Gateway work together in practice. Something that would make learning with NextWork even better is more guided examples around common errors and troubleshooting, especially for API deployment and integrations, so learners can more easily understand what to check when things don't work the first time.



Lambda functions

AWS Lambda is a serverless compute service that lets you run code in response to events without provisioning or managing servers. I'm using Lambda in this project to run backend logic for my API, automatically handling requests from API Gateway while scaling on demand and only charging for the execution time.

The code I added to my function retrieves user data from a DynamoDB table using a userId provided in the API request. It looks up the matching record, returns the data as a JSON response if found, and handles missing data or errors with appropriate HTTP status codes, allowing the Lambda function to act as a simple serverless API backend.



```
JS index.mjs > ...
1 // Import individual components from the DynamoDB client package
2 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
3 import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
4
5 const ddbClient = new DynamoDBClient({ region: 'us-east-1' });
6 const ddb = DynamoDBDocumentClient.from(ddbClient);
7
8 async function handler(event) {
9   const userId = event.queryStringParameters.userId;
10   const params = {
11     TableName: 'UserData',
12     Key: { userId }
13   };
14
15   try {
16     const command = new GetCommand(params);
17     const { Item } = await ddb.send(command);
18     if (Item) {
19       return {
20         statusCode: 200,
21         body: JSON.stringify(Item),
22         headers: { 'Content-Type': 'application/json' }
23       };
24     } else {
25       return {
26         statusCode: 404,
27         body: JSON.stringify({ message: "No user data found" }),
28         headers: { 'Content-Type': 'application/json' }
29       };
30     }
31   }
32 }
```



API Gateway

APIs are interfaces that let different applications communicate by sending requests and receiving responses. There are different types of APIs, like REST APIs (standard web APIs using HTTP methods), HTTP APIs (lighter, faster option for many use cases), and WebSocket APIs (real-time, two-way communication). My API is a REST API in Amazon API Gateway, which I'm using to connect user requests to my Lambda function.

Amazon API Gateway is a fully managed service that lets you create, publish, and manage APIs at scale. I'm using API Gateway in this project to expose my Lambda function as an HTTP endpoint, allowing users to send requests and receive responses from my serverless backend without managing any servers.

When a user makes a request to the API endpoint, Amazon API Gateway receives the request, passes it to the Lambda function for processing, and then returns the Lambda function's response back to the user, enabling seamless communication between the client and the serverless backend.



Successfully created REST API 'UserRequestAPI (mrgj9oue5e)'.

Create resource

/

Resource details

Path

/

Resource ID

mxyrtgpr1

Methods (0)

Delete

Create method

Method type

▲

Integration type

▼

Authorization

▼

API key

▼

No methods

No methods defined.

API actions

▼

Deploy API

Update documentation

Enable CORS



API Resources and Methods

An API is made up of resources, which represent specific objects or data that the API exposes, and each resource can support different actions through HTTP methods such as GET, POST, PUT, or DELETE.

Each resource consists of methods, which are the actions that can be performed on that resource using HTTP verbs such as GET, POST, PUT, and DELETE, defining how clients interact with the API.

I created a GET method for my API resource and integrated it with my Lambda function using Lambda proxy integration, so that incoming HTTP requests are passed directly to the function and the response is returned to the client automatically.



Create method

Method details

Method type

GET

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.


☐ **HTTP**
Integrate with an existing HTTP endpoint.


☐ **Mock**
Generate a response based on API Gateway mappings and transformations.


☐ **AWS service**
Integrate with an AWS Service.


☐ **VPC link**
Integrate with a resource that isn't accessible over the public internet.


☒ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

Response transfer mode [Info](#)

☒ **Buffered**
Wait to receive the complete response before beginning transmission.



API Deployment

A stage is a named environment in API Gateway that represents a specific version of your API (such as development, testing, or production) and provides a unique URL endpoint to access it. I deployed my API to the prod stage, which means this version is treated as the production-ready API that users can access publicly.

To visit my API, I opened the Invoke URL for the prod stage in API Gateway in my browser. The API displayed a “Missing Authentication Token” error because I accessed only the base stage URL (/prod) instead of calling a specific deployed resource and method (for example /prod/users), so API Gateway couldn’t match the request to a valid endpoint.



Deploy API



Create or select a stage where your API will be deployed. You can use the deployment history to revert or change the active deployment for a stage. [Learn more](#)

Stage

New stage



Stage name

prod



A new stage will be created with the default settings. Edit your stage settings on the **Stage** page.

Deployment description

[Cancel](#)

[Deploy](#)



API Documentation

For my project's extension, I am writing API documentation because it clearly explains how my API works what endpoints are available, which HTTP methods to use, what parameters are required, and what responses to expect making it easy for other developers (or future me) to use and test the API correctly. You can do this in API Gateway by generating and exporting the API's documentation in JSON format.

Once I prepared my documentation, I can publish it to API Gateway for a specific stage of my API. You have to publish your API documentation to a specific stage because each stage represents a deployed version of the API with its own configuration and endpoints. I used documentation version 1 and published it to the prod stage so the documentation accurately reflects the live, production-ready API.

My published and downloaded documentation showed me a complete, structured overview of my live API, including the API name (UserRequestAPI), the prod base URL, the available /users GET endpoint, the expected JSON responses, and the Lambda function integration used to retrieve user data. It also confirmed that the documentation was published as version 1 for the prod stage, clearly reflecting the production-ready configuration of my API and making it easy for others to understand how to use it correctly.



```
{
  "swagger": "2.0",
  "info": {
    "version": "2025-12-25T21:53:42Z",
    "title": "UserRequestAPI"
  },
  "host": "mrgj9oue5e.execute-api.us-east-1.amazonaws.com",
  "basePath": "/prod",
  "schemes": [ "https" ],
  "paths": {
    "/users": {
      "get": {
        "produces": [ "application/json" ],
        "responses": {
          "200": {
            "description": "200 response",
            "schema": {
              "$ref": "#/definitions/Empty"
            }
          }
        }
      },
      "x-amazon-apigateway-integration": {
        "type": "aws_proxy",
        "uri": "arn:aws:apigateway:us-east-1:lambda:path/2015-03-31/functions/arn:aws:lambda:us-east-1:006834263464:function:RetrieveUserData/invocations",
        "httpMethod": "POST",
        "responses": {
          "default": {
            "statusCode": "200"
          }
        },
        "passthroughBehavior": "when_no_match",
        "timeoutInMillis": 20000,
        "responseTransferMode": "BUFFERED",
        "contentHandling": "CONVERT_TO_TEXT"
      }
    }
  },
  "definitions": {
    "Empty": {
      "type": "object",
      "title": "Empty Schema"
    }
  }
}
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

